CSC 380 | Fundamentals of Database Systems
King Saud University

HOSPITAL MANAGEMENT SYSTEM

Phase 1 Report

Student Name	Student ID	Section
Alfarouq Alshukri	446109472	32714
Abdulmalik Alharbi	446104636	21959
Khalid Alhazlool	446102664	21959

Instructor: Dr. Amir Touir

Github: Alfarouq-Alshukri/CSC380-Hospital-Management-System

1. Project Description

I. Overview

We are developing a desktop Hospital Management System to manage patient, medical record, and doctor information. Hospital staff can use it to schedule appointments, track patient-doctor relationships, and manage patient admissions and room assignments.

II. Requirements

- **Patient** uses Patient_ID as its primary key and stores the patient's full name, date of birth, gender, and phone number.
- **Medical_Record** uses Record_ID as its primary key and stores blood type and the record creation date. Each record is associated with at most one patient, and each patient can have at most one record (1:1).
- **Doctor** uses Doctor_ID as its primary key and stores the doctor's full name, specialty, and phone number. Through TREATS (M:N relationship), a doctor may treat many patients, and a patient may be treated by many doctors.
- **Appointment** uses Appointment_ID as its primary key and stores the scheduled date and time and status. Each appointment is booked by one patient and attended by one doctor. Each patient and doctor may have many appointments (1 in both cases).
- **Room** uses Room_ID as its primary key and stores its location and type.
- **HAS_ADMISSION** is an M:N relationship between Patient and Room, with StartDate and EndDate as attributes (and partial keys). A patient may have multiple admissions over time, and a room may accommodate multiple patients across different admission periods.

III. Constraints

1. **Valid admission period:** StartDate is required. EndDate must be later than StartDate.

2. **Room occupancy:** Admission periods for the same room cannot overlap. A new admission may begin when the previous one ends. An admission without EndDate is considered ongoing.

3. **Patient admission:** A patient cannot have overlapping admissions. Once an admission ends, the patient may be admitted again.

2. Entity-Relationship (ER) Diagram

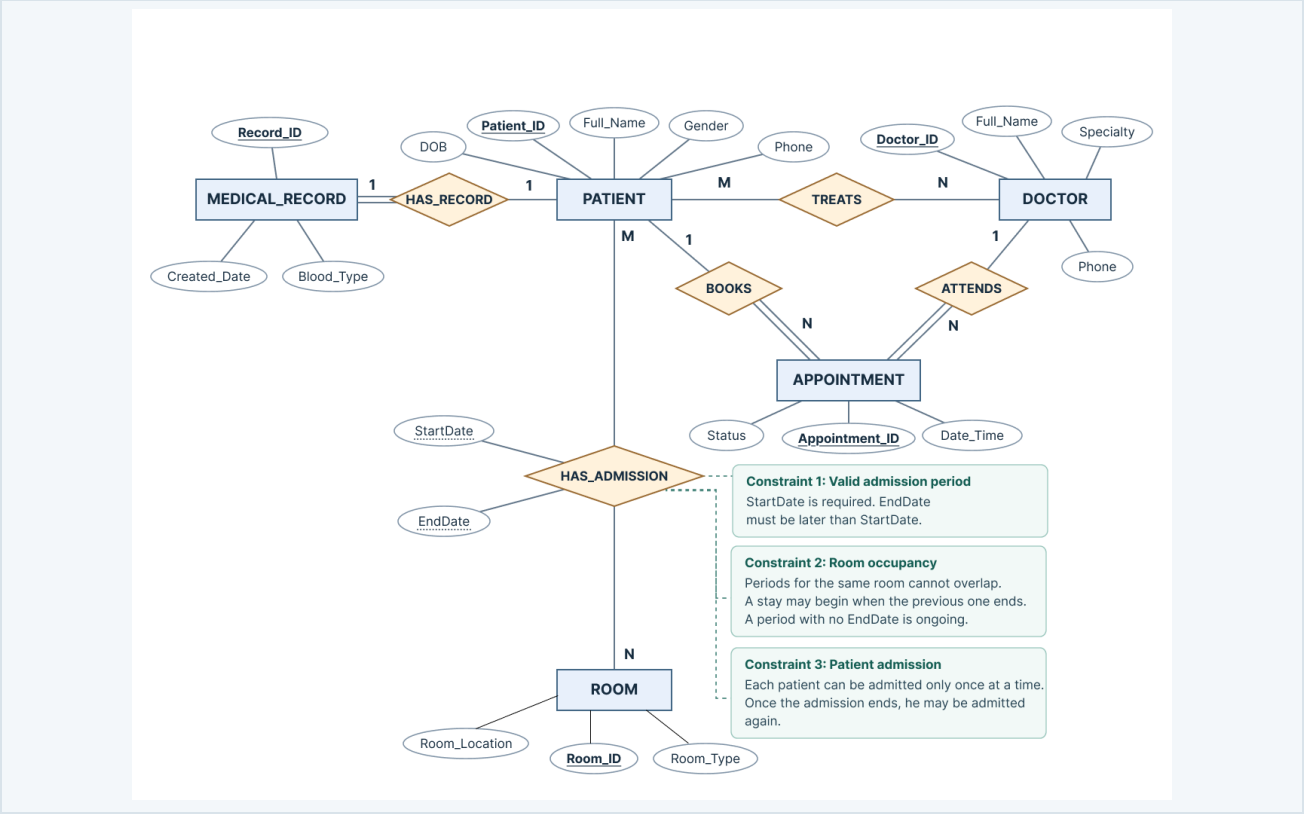


Figure 1. Conceptual Entity-Relationship Diagram

3. Relational Schema

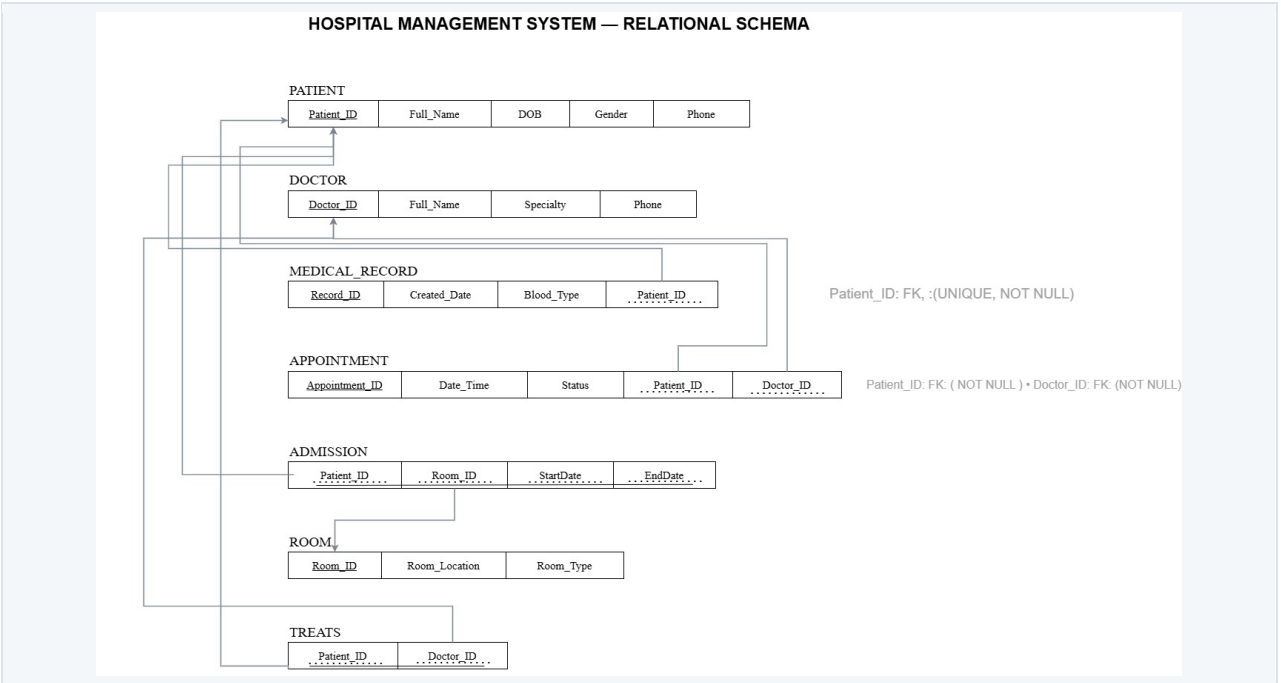


Figure 2. Mapped Relational Schema

4. SQL Schema

The database model was created in MySQL Workbench. The Forward Engineer option was used to generate the SQL schema from the model.

A. MySQL Workbench Diagram

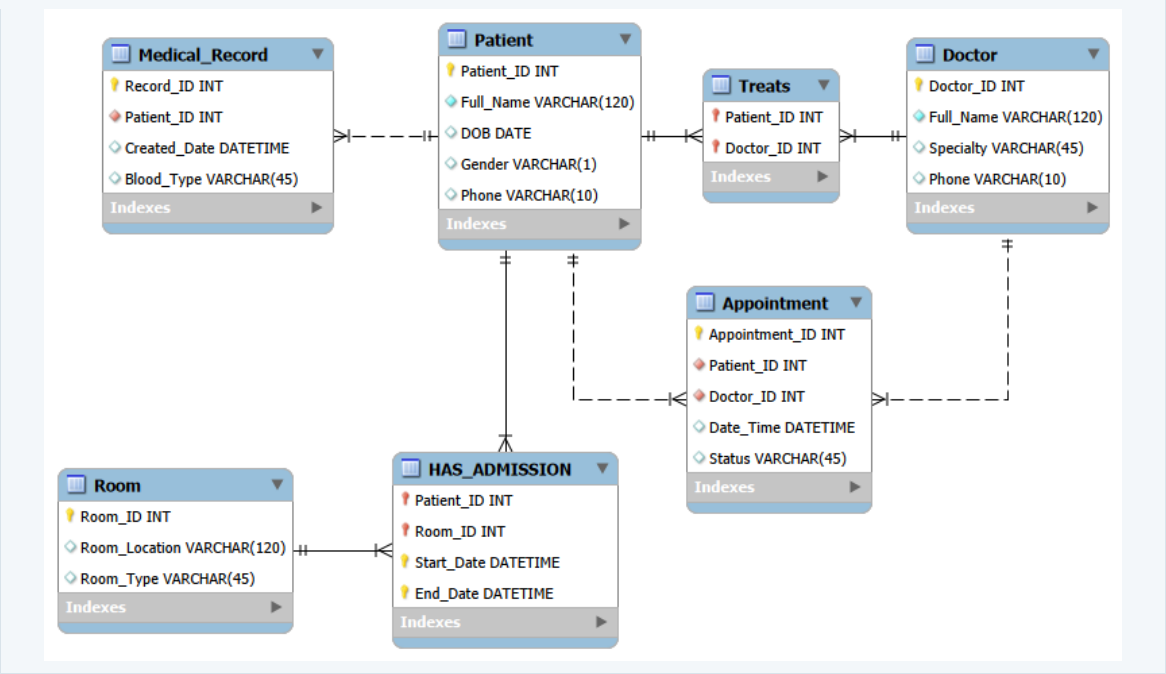


Figure 3. MySQL Workbench Database Model

B. Generated SQL Schema:

```
-- MySQL Script generated by MySQL Workbench
-- Thu Nov  5 16:46:15 2026
-- Model: New Model    Version: 1.0
-- MySQL Workbench Forward Engineering

SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE,
SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_
DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';

-- -----------------------------------------------------
-- Schema HOSPITAL
-- -----------------------------------------------------


-- -----------------------------------------------------
-- Schema HOSPITAL
-- -----------------------------------------------------
CREATE SCHEMA IF NOT EXISTS `HOSPITAL` DEFAULT CHARACTER SET utf8 ;
```

```sql
USE `HOSPITAL` ;

-- -----------------------------------------------------
-- Table `HOSPITAL`.`Patient`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `HOSPITAL`.`Patient` (
  `Patient_ID` INT NOT NULL,
  `Full_Name` VARCHAR(120) NOT NULL,
  `DOB` DATE NULL,
  `Gender` VARCHAR(1) NULL,
  `Phone` VARCHAR(10) NULL,
  PRIMARY KEY (`Patient_ID`))
ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `HOSPITAL`.`Medical_Record`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `HOSPITAL`.`Medical_Record` (
  `Record_ID` INT NOT NULL,
  `Patient_ID` INT NOT NULL,
  `Created_Date` DATETIME NULL,
  `Blood_Type` VARCHAR(45) NULL,
  PRIMARY KEY (`Record_ID`),
  UNIQUE INDEX `Patient_id_UNIQUE` (`Patient_ID` ASC) VISIBLE,
  CONSTRAINT `fk_Md_patient_id`
    FOREIGN KEY (`Patient_ID`)
    REFERENCES `HOSPITAL`.`Patient` (`Patient_ID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `HOSPITAL`.`Room`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `HOSPITAL`.`Room` (
  `Room_ID` INT NOT NULL,
  `Room_Location` VARCHAR(120) NULL,
  `Room_Type` VARCHAR(45) NULL,
  PRIMARY KEY (`Room_ID`))
ENGINE = InnoDB;


-- -----------------------------------------------------
```

```
-- Table `HOSPITAL`.`Doctor`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `HOSPITAL`.`Doctor` (
  `Doctor_ID` INT NOT NULL,
  `Full_Name` VARCHAR(120) NOT NULL,
  `Specialty` VARCHAR(45) NULL,
  `Phone` VARCHAR(10) NULL,
  PRIMARY KEY (`Doctor_ID`))
ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `HOSPITAL`.`HAS_ADMISSION`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `HOSPITAL`.`HAS_ADMISSION` (
  `Patient_ID` INT NOT NULL,
  `Room_ID` INT NOT NULL,
  `Start_Date` DATETIME NOT NULL,
  `End_Date` DATETIME NOT NULL,
  PRIMARY KEY (`Patient_ID`, `Room_ID`, `Start_Date`, `End_Date`),
  INDEX `adm_room_id_idx` (`Room_ID` ASC) VISIBLE,
  CONSTRAINT `adm_patient_id`
    FOREIGN KEY (`Patient_ID`)
    REFERENCES `HOSPITAL`.`Patient` (`Patient_ID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `adm_room_id`
    FOREIGN KEY (`Room_ID`)
    REFERENCES `HOSPITAL`.`Room` (`Room_ID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `HOSPITAL`.`Treats`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `HOSPITAL`.`Treats` (
  `Patient_ID` INT NOT NULL,
  `Doctor_ID` INT NOT NULL,
  PRIMARY KEY (`Patient_ID`, `Doctor_ID`),
  INDEX `Doctor_id_idx` (`Doctor_ID` ASC) VISIBLE,
  CONSTRAINT `T_patient_id`
    FOREIGN KEY (`Patient_ID`)
    REFERENCES `HOSPITAL`.`Patient` (`Patient_ID`)
```

```
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `T_doctor_id`
    FOREIGN KEY (`Doctor_ID`)
    REFERENCES `HOSPITAL`.`Doctor` (`Doctor_ID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `HOSPITAL`.`Appointment`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `HOSPITAL`.`Appointment` (
  `Appointment_ID` INT NOT NULL,
  `Patient_ID` INT NOT NULL,
  `Doctor_ID` INT NOT NULL,
  `Date_Time` DATETIME NULL,
  `Status` VARCHAR(45) NULL,
  PRIMARY KEY (`Appointment_ID`),
  INDEX `ap_patient_id_idx` (`Patient_ID` ASC) VISIBLE,
  INDEX `ap_doctor_id_idx` (`Doctor_ID` ASC) VISIBLE,
  CONSTRAINT `ap_patient_id`
    FOREIGN KEY (`Patient_ID`)
    REFERENCES `HOSPITAL`.`Patient` (`Patient_ID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `ap_doctor_id`
    FOREIGN KEY (`Doctor_ID`)
    REFERENCES `HOSPITAL`.`Doctor` (`Doctor_ID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;


SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
```

5. Technical Environment

Component	Technology / Purpose
Database and modeling	MySQL and MySQL Workbench.
Desktop application	Python for application logic, PySide6 for the interface, and mysql-connector-python for the database connection.
Development and collaboration	Visual Studio Code and GitHub.
Operating system	Windows.